

NEXAPAY

Plateforme fintech de traitement de paiements

CAHIER DES CHARGES FONCTIONNEL ET TECHNIQUE

Spécification de référence pour un système de paiement production-grade
Projet fil rouge Senior Backend Engineering - Java / Spring Boot

Document	Cahier des charges fonctionnel et technique
Version	1.0
Date	18 août 2026
Statut	Baseline de développement

Nom de travail modifiable avant publication publique.

0. Contrôle du document

0.1 Objet

Ce document définit le périmètre fonctionnel, l'architecture, les contraintes d'ingénierie, les exigences non fonctionnelles, les interfaces, les modèles de données, les stratégies de sécurité, de test, de déploiement et d'exploitation de NexaPay. Il doit servir de source de vérité pendant le développement et de support de démonstration technique lors d'entretiens.

0.2 Gestion des versions

Version	Date	Statut	Évolution
1.0	18/08/2026	Baseline	Spécification initiale complète
1.1	À venir	Prévu	Ajustements après conception détaillée / ADR
2.0	À venir	Prévu	Architecture distribuée et production cloud validées

0.3 Règles de gouvernance documentaire

- Toute décision structurante doit être tracée dans un ADR (Architecture Decision Record).
- Toute évolution d'API incompatible doit être versionnée et documentée.
- Toute exigence modifiée doit conserver son identifiant afin de préserver la traçabilité.
- Le README public résume le système ; ce cahier des charges contient la référence exhaustive.
- Les secrets, identifiants réels de prestataires et données personnelles ne doivent jamais figurer dans le dépôt Git.

0.4 Convention de priorité

Niveau	Signification
P0 - Critique	Indispensable à l'intégrité financière, la sécurité ou la disponibilité du service.
P1 - Haute	Nécessaire pour une version production crédible.
P2 - Moyenne	Amélioration importante mais non bloquante pour le MVP.
P3 - Basse	Extension ou optimisation ultérieure.

Table des matières

1. Résumé exécutif et vision	4
2. Contexte, objectifs et principes directeurs	5
3. Périmètre et parties prenantes	6
4. Architecture progressive et architecture cible	8
5. Exigences fonctionnelles	10
6. Cycle de vie des paiements et remboursements	12
7. Modèle de données et ledger	14
8. Sécurité, identité et conformité	15
9. Idempotence, webhooks et cohérence distribuée	17
10. Kafka et architecture événementielle	19
11. Moteur de fraude ML	21
12. API et contrats d'intégration	22
13. Cache, performance et résilience	23
14. Observabilité et exploitation	24
15. Stratégie de tests et qualité	25
16. CI/CD, infrastructure et Kubernetes	26
17. Exigences non fonctionnelles / SLO	28
18. Roadmap de développement par niveaux	29
19. Critères d'acceptation et Definition of Done	30
20. Risques, décisions et dette technique	31
21. Livrables du repository et documentation	32
22. Annexes techniques	33

1. Résumé exécutif et vision

Vision

Construire une plateforme de traitement de paiements crédible en production, permettant à des marchands d'accepter des paiements, effectuer des remboursements, recevoir des webhooks fiables, consulter un ledger auditable et soumettre chaque transaction à un moteur de risque/fraude. Le projet sert simultanément de produit démontrable et de laboratoire d'ingénierie backend avancée.

NexaPay ne cherche pas à reproduire l'intégralité d'un acteur comme Stripe. Le but est de couvrir un sous-ensemble cohérent, techniquement profond et exploitable : orchestration de paiements, intégrations externes, garanties transactionnelles, sécurité, événementiel, résilience, observabilité, automatisation de livraison et déploiement cloud.

1.1 Résultats attendus

- Une API Spring Boot stable, versionnée, documentée et sécurisée.
- Un modèle transactionnel empêchant les doubles débits et sur-remboursements.
- Un ledger immuable permettant l'audit et la réconciliation.
- Une gestion fiable des webhooks entrants et sortants.
- Une architecture événementielle utilisant Kafka seulement lorsque le domaine est stable.
- Un moteur de fraude intégré via un modèle ONNX/PMML versionné.
- Une chaîne CI/CD complète et un déploiement reproductible par IaC.
- Des métriques, logs et traces permettant de diagnostiquer les incidents.
- Des preuves de charge, résilience, sécurité et qualité de code.
- Une documentation d'architecture suffisante pour défendre chaque décision en entretien.

1.2 Critère de réussite global

Le projet est considéré abouti lorsqu'un environnement de production ou préproduction reproductible peut traiter un flux de paiement complet, résister aux doublons et aux pannes courantes, être observé de bout en bout, être déployé automatiquement et fournir des preuves de conformité aux exigences de ce document.

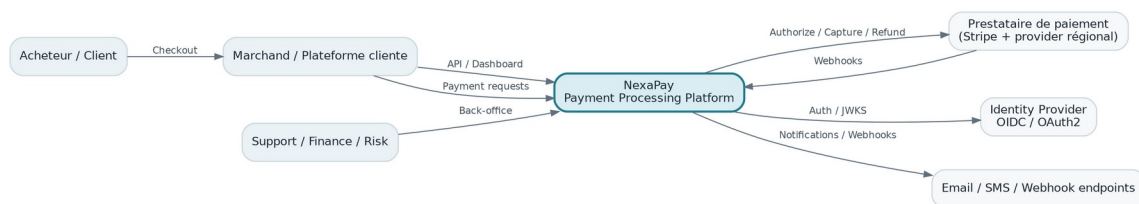


Figure 1 - Diagramme de contexte système.

2. Contexte, objectifs et principes directeurs

2.1 Problème métier simulé

Des marchands souhaitent intégrer une API unique pour initier, confirmer et rembourser des paiements, sans dépendre directement de la complexité d'un prestataire externe. NexaPay normalise les opérations de paiement, conserve une vérité métier locale, applique des règles de risque et garantit des notifications fiables vers les systèmes marchands.

2.2 Objectifs métier

ID	Objectif	Priorité
OBJ-01	Permettre l'onboarding et la configuration d'un marchand.	P1
OBJ-02	Traiter un paiement de bout en bout avec prévention des doublons.	P0
OBJ-03	Gérer remboursement partiel et total sans dépasser le montant capturé.	P0
OBJ-04	Fournir un historique auditable et réconciliable.	P0
OBJ-05	Notifier les systèmes marchands de façon fiable.	P1
OBJ-06	Évaluer le risque de fraude en temps réel.	P1
OBJ-07	Supporter la montée en charge et les pannes de dépendances.	P1
OBJ-08	Être déployable de manière automatisée, observable et sécurisée.	P1

2.3 Principes directeurs d'ingénierie

- PostgreSQL est la source de vérité des états financiers et transactionnels.
- La correction et l'intégrité financière priment sur la disponibilité d'une opération mutative.
- Toute commande à effet financier doit être idempotente.
- Les événements ne remplacent pas les invariants transactionnels locaux.
- Le système commence en modular monolith ; les microservices sont introduits par décision explicite.
- Chaque dépendance externe est traitée comme lente, faillible et potentiellement incohérente.
- Les secrets et les données sensibles sont minimisés, chiffrés ou tokenisés.
- Toute fonctionnalité critique doit être observable et testable sous échec.
- Aucune technologie n'est ajoutée uniquement pour enrichir le CV ; elle doit résoudre un problème identifié.

3. Périmètre et parties prenantes

3.1 Acteurs

Acteur	Responsabilité / besoin principal
Marchand	Intègre l'API, initie les paiements, remboursements et reçoit les événements.
Client final	Effectue un achat chez le marchand ; n'interagit pas nécessairement directement avec NexaPay.
Administrateur marchand	Gère clés API, endpoints webhook, utilisateurs et paramètres.
Finance / Support	Recherche une transaction, vérifie le ledger, initie ou suit des opérations autorisées.
Risk analyst	Analyse les décisions de fraude et ajuste les politiques non-ML.
SRE / Exploitant	Surveille SLO, incidents, déploiements, capacité et sauvegardes.
Prestataire de paiement	Exécute autorisations, captures, remboursements et envoie des webhooks.
Identity Provider	Authentifie les utilisateurs humains et, selon le scénario, les clients machine-to-machine.

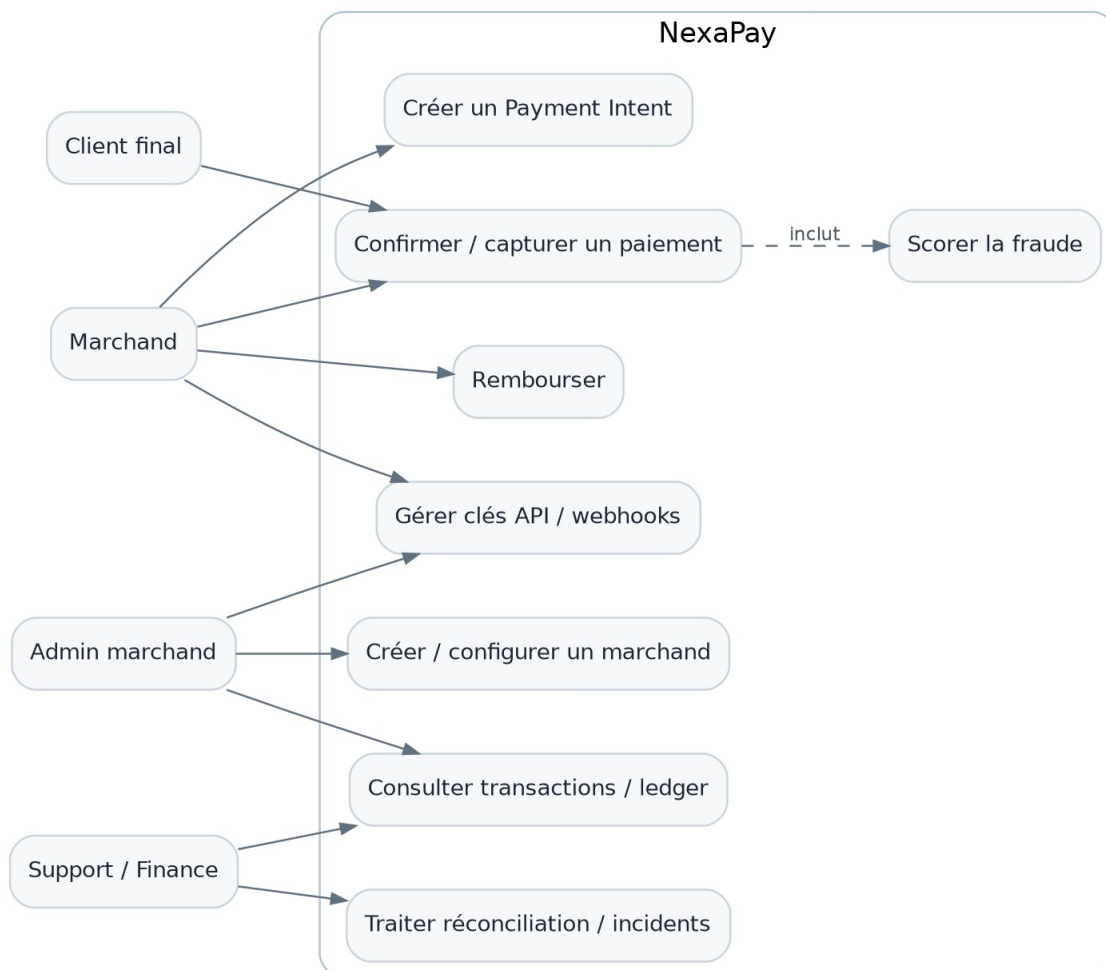


Figure 2 - Vue synthétique des cas d'utilisation.

3.2 Dans le périmètre

- Onboarding marchand et statut du compte.
- Gestion de clés API et scopes.

- Payment Intent, confirmation, autorisation/capture.
- Remboursement partiel / total.
- Intégration Stripe en sandbox et abstraction multi-provider.
- Webhooks entrants signés et dédoublés.
- Webhooks sortants vers le marchand avec retry et DLQ.
- Ledger en double entrée simplifié.
- Fraud scoring temps réel via modèle pré-entraîné.
- Audit log, recherche et réconciliation.
- Kafka, Redis, observabilité, CI/CD, Terraform et Kubernetes dans les phases avancées.

3.3 Hors périmètre initial

- Acquisition bancaire réelle ou statut réglementaire de prestataire de services de paiement.
- Stockage direct des numéros complets de carte ou CVV.
- Chargeback/dispute management complet.
- Conversion de devises et treasury avancé.
- Paiement récurrent/subscriptions dans le MVP.
- Application mobile grand public.
- Entraînement complet d'un modèle ML propriétaire à partir de données sensibles réelles.

4. Architecture progressive et architecture cible

4.1 Phase d'apprentissage : modular monolith

Le premier objectif est de maîtriser le domaine et les garanties locales sans introduire prématurément la complexité distribuée. Les modules doivent avoir des frontières explicites, peu de dépendances cycliques et communiquer via des interfaces métier.

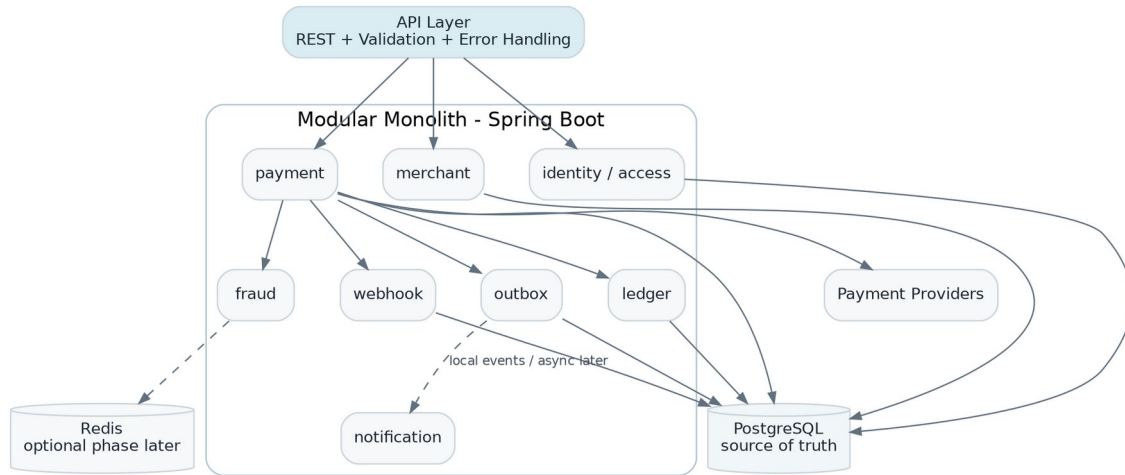


Figure 3 - Architecture de départ : modular monolith.

Règles de modularité

- Un module ne lit pas directement les tables privées d'un autre module.
- Les dépendances autorisées sont explicites et vérifiables par tests d'architecture (ex. ArchUnit).
- Les événements de domaine locaux ne sont pas synonymes d'événements Kafka.
- La transaction métier reste locale au monolithe tant que possible.
- Les modules payment et ledger sont considérés critiques et fortement testés.

4.2 Architecture cible distribuée

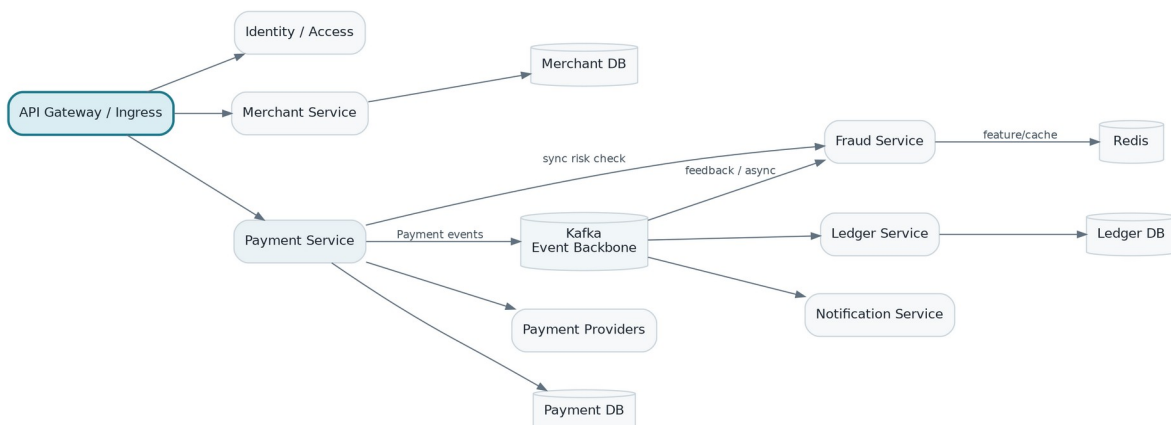


Figure 4 - Architecture cible après extractions justifiées.

Critères d'extraction en microservice

Critère	Question à valider avant extraction
Scalabilité	Le module a-t-il un profil de charge sensiblement différent ?
Isolation	Une panne doit-elle être contenue sans affecter le cœur paiement ?
Cycle de livraison	Le module nécessite-t-il un déploiement indépendant fréquent ?

Données	La frontière de propriété des données est-elle suffisamment stable ?
Organisation	Existe-t-il une raison d'autonomie de service, même dans un projet individuel simulé ?
Complexité	Le bénéfice dépasse-t-il les coûts réseau, observabilité, cohérence et opérations ?

5. Exigences fonctionnelles

5.1 Gestion des marchands

ID	Exigence	Priorité
FR-MER-001	Créer un compte marchand avec identifiant unique, nom, statut et métadonnées.	P1
FR-MER-002	Activer, suspendre ou fermer un marchand selon une machine à états explicite.	P1
FR-MER-003	Créer, révoquer et faire tourner les clés API sans jamais stocker la clé secrète en clair.	P0
FR-MER-004	Associer des scopes aux clés API et contrôler les permissions à chaque requête.	P0
FR-MER-005	Configurer un ou plusieurs endpoints webhook et un secret de signature.	P1
FR-MER-006	Afficher les métriques de paiement essentielles du marchand.	P2

5.2 Paiements

ID	Exigence	Priorité
FR-PAY-001	Créer un Payment Intent avec montant entier en minor units, devise, référence externe et metadata.	P0
FR-PAY-002	Exiger une clé d'idempotence pour toute création ou confirmation financière.	P0
FR-PAY-003	Refuser les montants nuls, négatifs ou hors limites configurées.	P0
FR-PAY-004	Confirmer un Payment Intent et exécuter la politique de fraude avant capture.	P0
FR-PAY-005	Supporter authorize+capture immédiat et, en extension, autorisation puis capture séparée.	P1
FR-PAY-006	Conserver provider, provider reference, codes d'erreur normalisés et timestamps.	P0
FR-PAY-007	Exposer une ressource canonique permettant au marchand de relire l'état final.	P0
FR-PAY-008	Garantir qu'un replay d'une même commande ne crée pas une nouvelle opération financière.	P0

5.3 Remboursements

ID	Exigence	Priorité
FR-REF-001	Créer un remboursement total ou partiel sur un paiement capturé.	P0
FR-REF-002	Empêcher la somme des remboursements réussis et en cours de dépasser le montant remboursable.	P0
FR-REF-003	Rendre la création de remboursement idempotente.	P0
FR-REF-004	Gérer les réponses asynchrones ou différées du prestataire.	P1
FR-REF-005	Produire les écritures de ledger et événements associés.	P0

5. Exigences fonctionnelles - suite

5.4 Webhooks entrants

ID	Exigence	Priorité
FR-WHI-001	Vérifier la signature cryptographique et la fenêtre temporelle du webhook fournisseur.	P0
FR-WHI-002	Dédupliquer sur l'identifiant provider_event_id ou équivalent.	P0
FR-WHI-003	Conserver un hash/payload minimal nécessaire à l'audit sans données sensibles inutiles.	P1
FR-WHI-004	Appliquer les transitions d'état de manière transactionnelle et monotone.	P0
FR-WHI-005	Répondre rapidement 2xx après persistance sûre ; déléguer le reste à l'asynchrone.	P1

5.5 Webhooks sortants

ID	Exigence	Priorité
FR-WHO-001	Signer chaque webhook marchand avec un secret par endpoint.	P0
FR-WHO-002	Livrer avec un event_id stable, type, version, timestamp et payload documenté.	P1
FR-WHO-003	Réessayer avec backoff exponentiel et jitter en cas d'échec transitoire.	P1
FR-WHO-004	Arrêter/reclasser les échecs permanents et diriger vers une DLQ après seuil.	P1
FR-WHO-005	Permettre la relivraison manuelle autorisée depuis le back-office.	P2

5.6 Recherche, audit et réconciliation

- Recherche par payment_id, merchant_id, external_reference, provider_reference et période.
- Affichage de la chronologie des changements d'état.
- Consultation des écritures de ledger liées.
- Journal d'audit séparant acteur, action, ressource, décision d'autorisation, résultat et trace_id.
- Job de réconciliation comparant les opérations locales avec les exports/API du provider et signalant les écarts.
- Export CSV/JSON réservé aux rôles autorisés et journalisé.

6. Cycle de vie des paiements et remboursements

6.1 Flux nominal de paiement

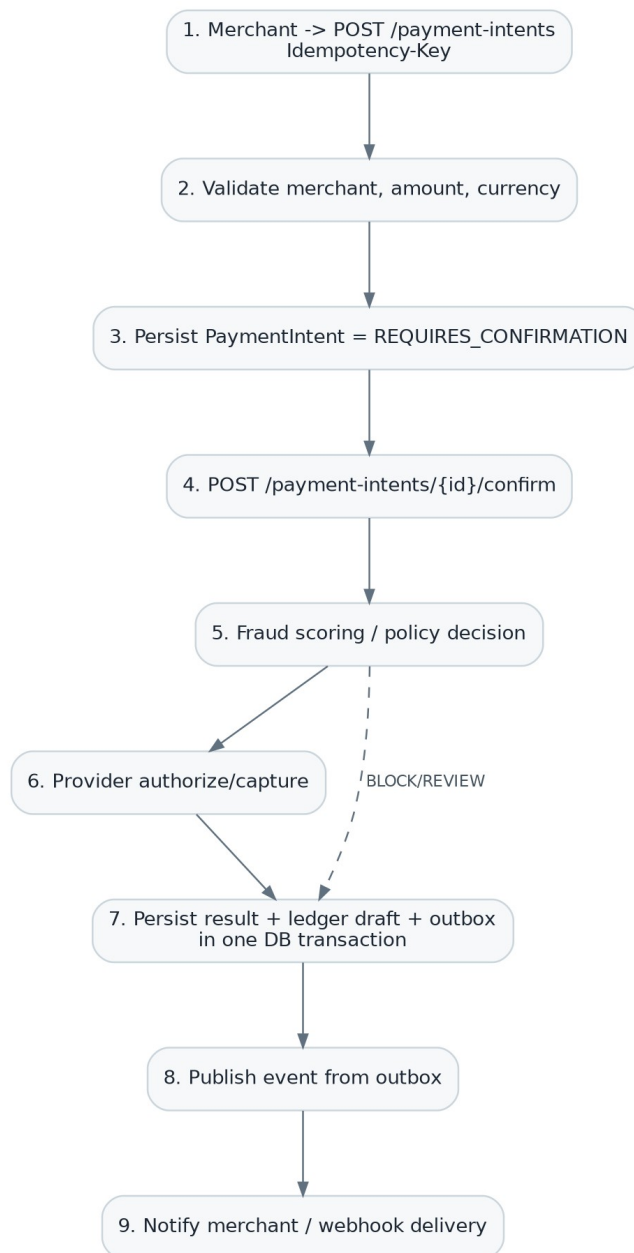


Figure 5 - Séquence fonctionnelle d'un paiement.

6.2 Machine à états du paiement



Figure 6 - Machine à états simplifiée du Payment Intent / Payment.

Invariants de transition

- Un paiement SUCCEEDED ne repasse jamais à PROCESSING ou FAILED.
- Une capture ne peut pas excéder le montant autorisé.
- Un remboursement ne peut viser qu'un paiement capturé avec un solde remboursable strictement positif.
- Un événement fournisseur ancien ne doit pas écraser un état plus avancé sans règle explicite de réconciliation.
- Les transitions sont contrôlées au domaine et renforcées par des contraintes/versions en base.

6.3 Flux de remboursement

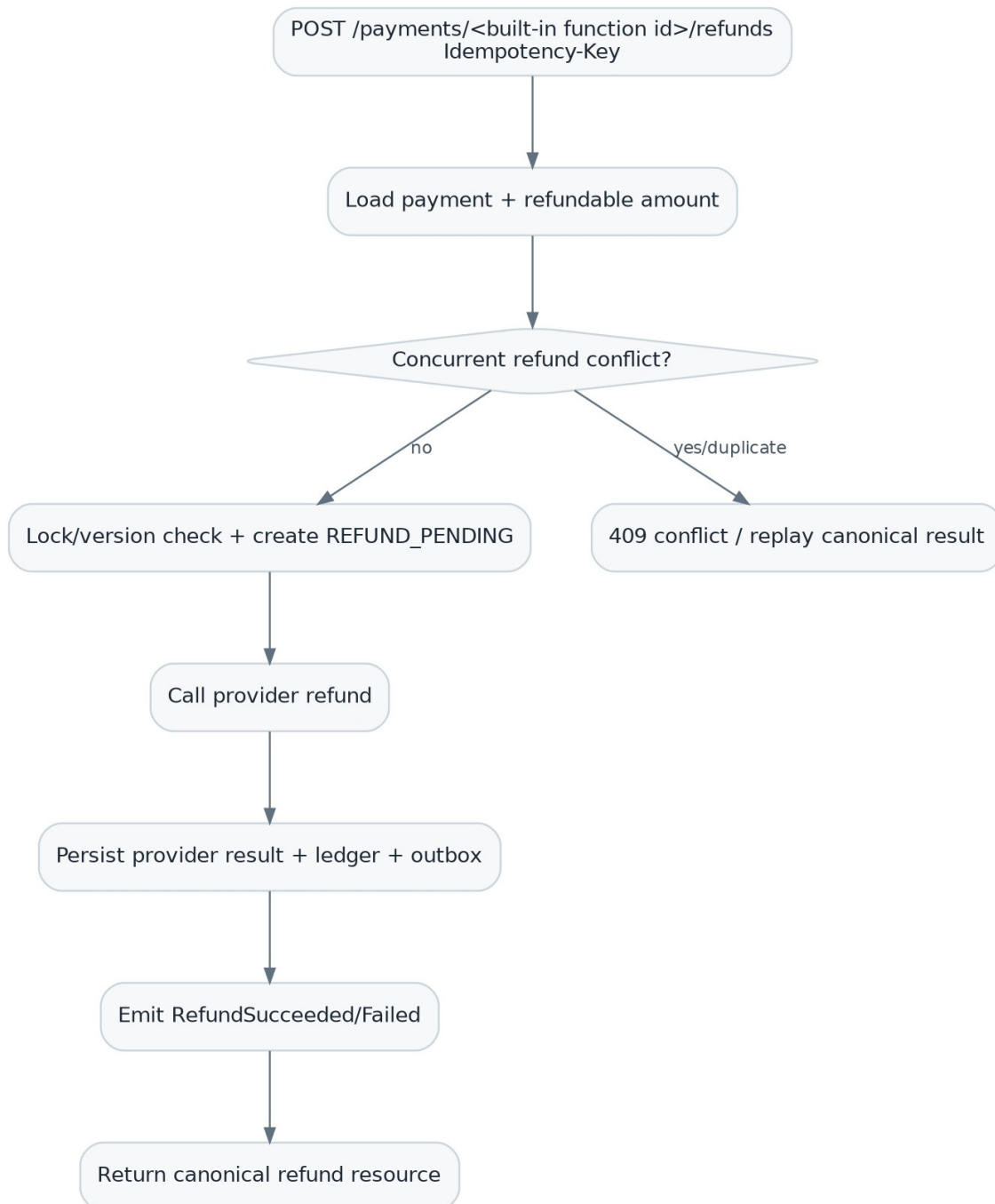


Figure 7 - Séquence de remboursement avec contrôle de concurrence.

7. Modèle de données et ledger

7.1 Modèle conceptuel

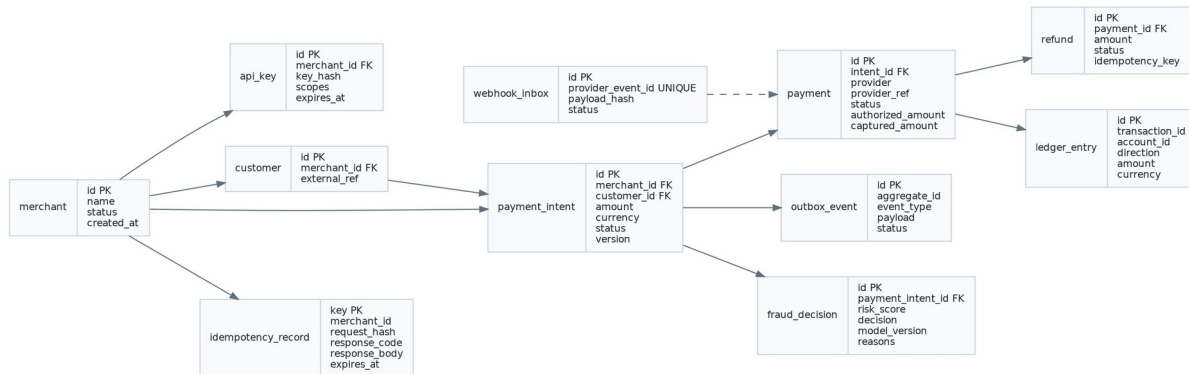


Figure 8 - ERD simplifié des entités principales.

7.2 Conventions de données

- Les montants sont stockés en integer/long de minor units ; jamais en floating point.
- La devise suit un code ISO approprié et est stockée explicitement sur chaque agrégat financier.
- Tous les timestamps sont persistés en UTC et exposés en ISO 8601.
- Les identifiants externes fournis par le marchand ne remplacent pas les identifiants internes.
- Toute table critique dispose de created_at, updated_at et, si nécessaire, d'une version optimiste.
- Les contraintes UNIQUE protègent les invariants contre les courses que l'application seule ne peut garantir.

7.3 Ledger en double entrée simplifié

Chaque mouvement financier crée un lot d'écritures équilibrées. Le ledger est append-only : une correction est réalisée par une écriture compensatoire, jamais par modification destructive d'une écriture historisée.

Exemple paiement 100 000 XOF	Débit	Crédit
Compte acquéreur / clearing	100 000	-
Compte marchand	-	98 000
Compte frais plateforme	-	2 000

Invariants du ledger

- Somme des débits = somme des crédits pour chaque ledger_transaction.
- Une transaction de ledger est immuable après commit.
- Les écritures référencent l'événement métier source.
- Une contrainte d'unicité empêche de comptabiliser deux fois le même événement métier.
- Les soldes dérivés peuvent être calculés ou matérialisés, mais les écritures restent la source d'audit.

8. Sécurité, identité et conformité

8.1 Modèle d'authentification



Figure 9 - Modèle d'authentification et d'autorisation.

8.2 Mécanismes attendus

Domaine	Mesures minimales
Utilisateurs humains	OIDC, MFA via IdP, sessions courtes, RBAC, journalisation des actions sensibles.
Machine-to-machine	API keys hashées ou OAuth2 client credentials, scopes minimaux, rotation/révocation.
Transport	TLS obligatoire ; aucun endpoint HTTP non chiffré en production.
Secrets	Secrets Manager/Vault/Kubernetes secrets chiffrés ; jamais dans Git ou images Docker.
Données	Minimisation, chiffrement au repos, tokenisation des références sensibles, rétention maîtrisée.
Webhooks	Signature HMAC/asymétrique selon provider, timestamp, protection replay, secret par endpoint.
API	Validation stricte, rate limiting, quotas, erreurs non verbeuses, headers de sécurité.
Administration	Least privilege, séparation des rôles, audit obligatoire, opérations sensibles re-confirmées.

8.3 Principes de conformité

Important

Le projet doit éviter de traiter ou stocker des données carte complètes. L'intégration privilégie les tokens et composants hébergés du prestataire. Toute référence à PCI DSS dans le projet doit être présentée comme une démarche de réduction de périmètre et de bonnes pratiques, pas comme une certification réelle.

8.4 Threat model minimal

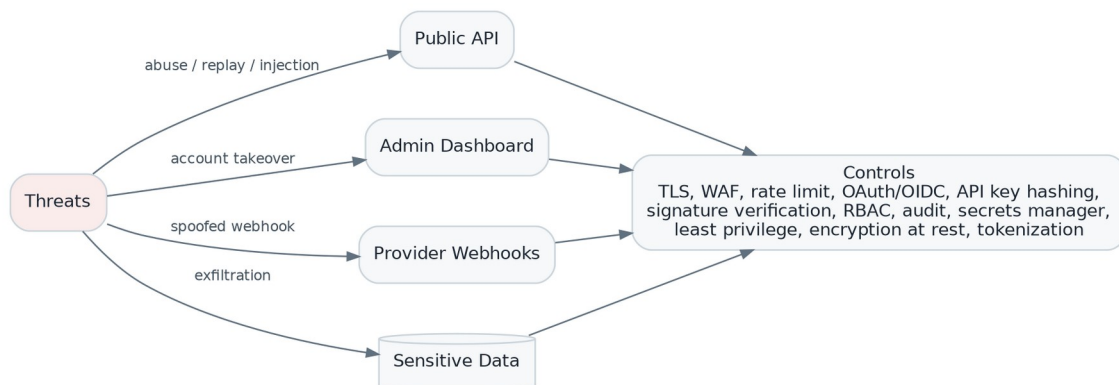


Figure 10 - Vue synthétique des principales surfaces de menace.

Scénarios à tester

- Replay d'une requête financière avec même et différente Idempotency-Key.

- Falsification d'un webhook fournisseur.
- Utilisation d'une clé API révoquée.
- Escalade de privilège entre marchands (IDOR).
- Injection SQL / payloads invalides / mass assignment.
- Brute-force ou abus de création de Payment Intents.
- Exposition involontaire de données sensibles dans logs et traces.
- Compromission d'un secret de webhook et procédure de rotation.

9. Idempotence, webhooks et cohérence distribuée

9.1 Idempotence API

Pour toute commande mutative financière, le client fournit une Idempotency-Key unique dans le périmètre du marchand. Le serveur calcule un fingerprint canonique de la requête. Si la clé existe avec le même fingerprint, la réponse canonique antérieure est rejouée ; si le fingerprint diffère, la requête est rejetée en conflit.

```
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
```

```
UNIQUE (merchant_id, idempotency_key)  
request_hash = SHA-256(canonical_request)
```

Cas de concurrence à couvrir

Cas	Comportement attendu
Deux requêtes simultanées, même clé et même payload	Une seule exécution métier ; l'autre attend/rejoue la ressource canonique.
Même clé, payload différent	409 Conflict ou erreur métier explicite.
Première exécution crash après commit	Le retry retrouve la réponse/ressource persistée.
Première exécution échoue avant effet externe	La politique définit si la clé peut être rejouée ou reste associée à l'échec.
Clé expirée	Nouvelle opération possible seulement selon politique documentée et après TTL sûr.

9.2 Webhooks entrants idempotents

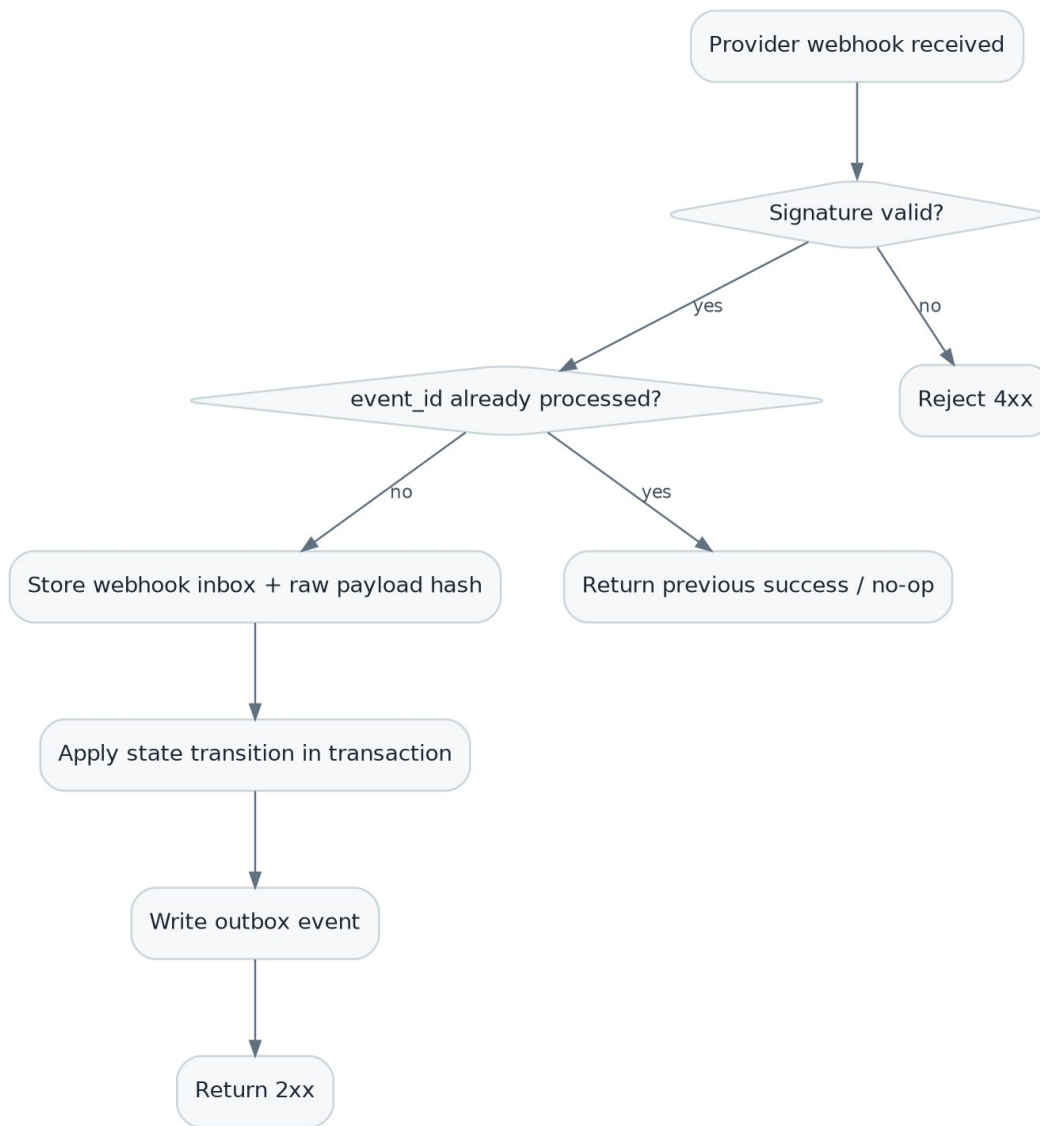


Figure 11 - Traitement idempotent d'un webhook fournisseur.

9.3 Transactional Outbox



Figure 12 - Transactional Outbox pour supprimer le dual-write naïf.

Le Payment Service ne publie pas directement un événement Kafka dans la même méthode qui modifie PostgreSQL en supposant une atomicité distribuée. Il écrit l'état métier et l'événement outbox dans la même transaction locale. Un publisher séparé livre ensuite vers Kafka avec retry et déduplication côté consommateurs.

10. Kafka et architecture événementielle

10.1 Objectifs

- Découpler les traitements non bloquants du chemin critique du paiement.
- Permettre à Ledger, Notification, Reconciliation et Fraud feedback de consommer les événements indépendamment.
- Préserver l'ordre là où il est requis grâce à une clé de partition stable (ex. payment_id ou merchant_id selon le besoin).
- Accepter une sémantique at-least-once et rendre les consommateurs idempotents.
- Versionner les contrats d'événements et tester leur compatibilité.

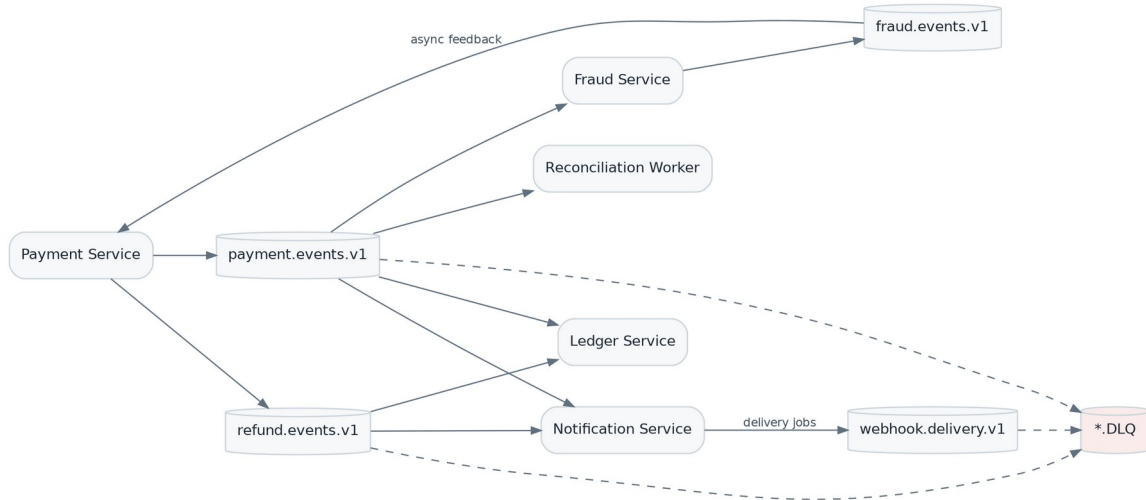


Figure 13 - Topologie logique des topics et consommateurs.

10.2 Catalogue initial des événements

Type	Producteur	Consommateurs	Clé suggérée
PaymentIntentCreated	Payment	Fraud, Analytics	payment_intent_id
PaymentSucceeded	Payment	Ledger, Notification, Reconciliation	payment_id
PaymentFailed	Payment	Notification, Analytics	payment_id
RefundSucceeded	Payment	Ledger, Notification	payment_id
FraudDecisionRecorded	Fraud	Payment/Analytics	payment_intent_id
WebhookDeliveryRequested	Notification	Webhook worker	merchant_id

10.3 Format d'enveloppe

```
{
  "eventId": "evt_...",
  "eventType": "payment.succeeded",
  "eventVersion": 1,
  "occurredAt": "2026-08-18T14:30:00Z",
  "aggregateId": "pay_...",
  "merchantId": "mrc_...",
  "traceId": "...",
  "data": { ... }
}
```

Règles consommateurs

- Dédupliquer par eventId dans un inbox/processed_events store lorsque l'effet n'est pas naturellement idempotent.
- Committer l'offset après persistance de l'effet métier.

- Utiliser retry limité pour les erreurs transitoires et DLQ pour les événements poison.
- Exposer consumer lag, taux de retry et taille de DLQ comme métriques.

11. Moteur de fraude ML

11.1 Objectif

Le moteur de fraude fournit un score de risque et une décision ALLOW / REVIEW / BLOCK avant l'appel de capture lorsque le temps de réponse le permet. Il ne doit pas transformer le projet backend en projet de data science : le modèle est pré-entraîné et servi comme artefact versionné via ONNX ou PMML.

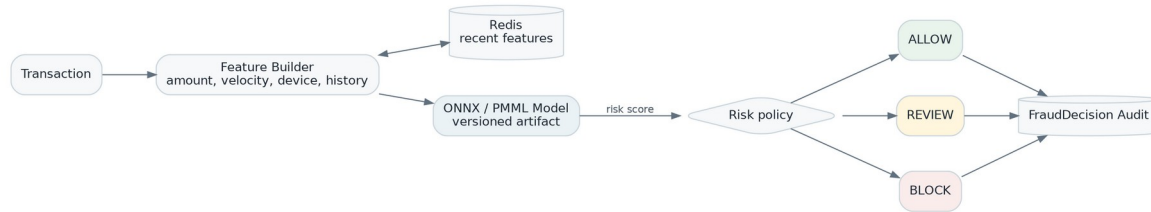


Figure 14 - Pipeline de scoring fraude en temps réel.

11.2 Features candidates

Catégorie	Exemples
Transaction	Montant, devise, heure, type de marchand, fréquence.
Velocity	Nombre et montant cumulé sur fenêtres glissantes par client/device/IP.
Device / réseau	Device fingerprint pseudonymisé, pays/zone, incohérence géographique.
Historique	Taux d'échec antérieur, nouveaux moyens de paiement, âge du compte.
Marchand	Profil de risque, catégorie, anomalies par rapport au baseline.
Provider feedback	Codes de refus, authentification forte, signaux retournés par le provider.

11.3 Contrat de décision

```

{
  "paymentIntentId": "pi_...",
  "riskScore": 0.87,
  "decision": "REVIEW",
  "modelVersion": "fraud-v1.3.0",
  "featureVersion": "features-v2",
  "reasons": ["HIGH_AMOUNT", "NEW_DEVICE", "HIGH_VELOCITY"]
}

```

11.4 Exigences spécifiques

- Le modèle et les features sont versionnés et leur version est auditée avec chaque décision.
- Le service définit un timeout strict ; il ne doit pas bloquer indéfiniment le paiement.
- La stratégie de dégradation est configurable : fail-open pour certains marchands à faible risque ou fail-closed/review pour les scénarios critiques.
- Redis peut accélérer les features de vélocité, mais l'indisponibilité du cache doit avoir un comportement déterministe.
- Les décisions doivent être explicables au niveau règles/reasons sans prétendre fournir une explication complète du modèle ML.

12. API et contrats d'intégration

12.1 Conventions REST

- Base path versionné : /v1.
- JSON UTF-8 ; Content-Type application/json.
- Identifiants opaques avec préfixes lisibles (mrc_, pi_, pay_, ref_, evt_).
- Montants en minor units et devise explicite.
- Pagination par cursor pour les grandes collections ; offset acceptable pour pages internes limitées.
- Correlation/trace ID retourné dans les réponses et logs.
- Codes HTTP cohérents : 200/201/202, 400 validation, 401 auth, 403 authorization, 404, 409 conflit/idempotence, 422 règle métier, 429 rate limit, 5xx dépendances/serveur.
- OpenAPI généré et vérifié dans CI.

12.2 Endpoints principaux

Méthode	Endpoint	Objet
POST	/v1/merchants	Créer un marchand.
POST	/v1/merchants/{id}/api-keys	Créer une clé API.
DELETE	/v1/api-keys/{id}	Révoquer une clé API.
POST	/v1/payment-intents	Créer une intention de paiement.
GET	/v1/payment-intents/{id}	Consulter l'état canonique.
POST	/v1/payment-intents/{id}/confirm	Confirmer et lancer le traitement.
POST	/v1/payments/{id}/refunds	Créer un remboursement.
GET	/v1/payments/{id}/refunds	Lister les remboursements.
POST	/v1/webhook-endpoints	Créer un endpoint marchand.
POST	/v1/webhook-events/{id}/redeliver	Relivrer un événement autorisé.
GET	/v1/ledger/transactions/{id}	Consulter la transaction de ledger.
GET	/v1/audit-events	Recherche d'audit réservée aux rôles autorisés.

12.3 Format d'erreur standardisé

```
{
  "type": "https://errors.nexapay.dev/payment-state-conflict",
  "title": "Payment state conflict",
  "status": 409,
  "code": "PAYMENT_STATE_CONFLICT",
  "detail": "Payment cannot be refunded in current state",
  "traceId": "...",
  "errors": []
}
```

12.4 Versioning et compatibilité

Les breaking changes nécessitent une nouvelle version majeure d'API ou une stratégie de compatibilité. Les événements Kafka possèdent un eventVersion indépendant. Les consommateurs doivent tolérer l'ajout de champs non obligatoires.

13. Cache, performance et résilience

13.1 Usage de Redis

Usage	Justification	Règle
Feature cache fraude	Fenêtres de vélocité et signaux récents.	TTL court ; comportement défini en cas de cache miss/down.
Configuration marchand	Lecture fréquente, modification rare.	Cache-aside + invalidation sur mise à jour.
Rate limiting	Limiter abus par merchant/key/IP.	Algorithme documenté ; ne pas utiliser Redis comme vérité financière.
Idempotency metadata - option	Accélération de replay non critique.	La garantie durable reste en base si nécessaire.

13.2 Résilience des dépendances

Dépendance	Timeout	Retry	Fallback
Payment provider	Court et explicite	Seulement opérations sûres/idempotentes	Retour PROCESSING/UNKNOWN si résultat ambigu ; réconciliation.
Fraud service	Très court	Limité	Fail-open/review/fail-closed selon policy.
Kafka	Non bloquant chemin API	Publisher retry/outbox	Événement reste en outbox.
Redis	Très court	Rare	Recalcul, cache miss ou policy dégradée.
Notification provider	Hors chemin critique	Backoff + DLQ	Incident visible sans annuler le paiement.

13.3 Scénarios de panne obligatoires

- Provider timeout après requête potentiellement acceptée.
- PostgreSQL devient lent ou connection pool saturé.
- Kafka indisponible pendant plusieurs minutes.
- Redis indisponible pendant scoring fraude.
- Un pod payment meurt après commit DB mais avant réponse HTTP.
- Webhook marchand retourne 500 ou timeout pendant plusieurs heures.
- Consumer Kafka traite deux fois le même événement.
- Deux remboursements concurrents ciblent le même solde remboursable.

13.4 Stratégies

- Timeouts à chaque appel réseau ; jamais de timeout infini.
- Retries uniquement lorsque l'opération est sûre ou rendue idempotente.
- Exponential backoff + jitter pour limiter les thundering herds.
- Circuit breaker sur dépendances adaptées ; bulkheads pour isoler les pools critiques.
- Queue/DLQ pour les échecs asynchrones persistants.
- Réconciliation pour les états provider ambigus au lieu d'inventer un succès/échec.

14. Observabilité et exploitation

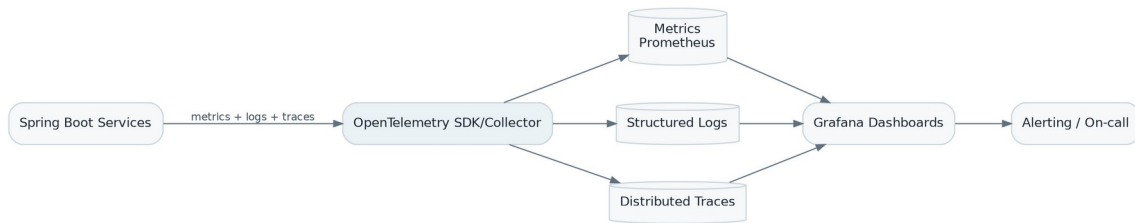


Figure 15 - Pipeline d'observabilité.

14.1 Signaux obligatoires

Signal	Exemples
Metrics	request_rate, error_rate, p95/p99 latency, DB pool usage, provider latency, payment success rate, Kafka lag, DLQ depth.
Logs	JSON structurés : timestamp, level, service, traceId, merchantId pseudonymisé, resourceId, eventType, errorCode.
Traces	API -> Payment -> Fraud -> Provider ; propagation via Kafka avec trace context.
Business metrics	Volume de paiements, taux d'échec, remboursements, fraude bloquée, webhook delivery success.
Audit	Actions administratives, changements de configuration, clés, remboursements manuels, relivraisons.

14.2 SLI / SLO proposés pour la démo production

Les valeurs ci-dessous sont des objectifs de conception du projet, pas des statistiques observées.

Indicateur	Objectif projet
Disponibilité API lecture	>= 99,9 % mensuel en environnement de démonstration stable.
Disponibilité commandes paiement	>= 99,5 % hors indisponibilité du provider externe.
Latence GET p95	< 200 ms hors dépendance externe.
Latence création Payment Intent p95	< 300 ms hors fraude/provider.
Webhook marchand	>= 99 % livré en moins de 5 min si endpoint disponible.
RPO base transactionnelle	<= 5 min ; idéalement proche de 0 via managed DB/replication.
RTO démonstration	<= 30 min pour reprise d'un incident infrastructure majeur.

14.3 Runbooks minimaux

- Provider payment timeout / état inconnu.
- Kafka outage / backlog outbox.
- DLQ growth.
- DB saturation / slow queries.
- Webhook delivery failure massif.
- Fraud service unavailable.
- Compromission/rotation d'un secret.
- Rollback d'un déploiement défectueux.

15. Stratégie de tests et qualité

15.1 Pyramide de tests

Niveau	But	Outils / approche
Unitaires	Invariants domaine, politiques, mapping d'erreurs.	JUnit, AssertJ, Mockito limité aux frontières.
Intégration	Repository, migrations, transactions, locks, Kafka/Redis.	Testcontainers PostgreSQL/Kafka/Redis.
API	Validation, sécurité, idempotence, Problem Details.	Spring Boot Test, RestAssured/WebTestClient.
Contrats	Compatibilité provider abstrait, OpenAPI et événements.	Contract tests / schema tests.
End-to-end	Paiement sandbox complet jusqu'au webhook marchand.	Environnement staging isolé.
Charge	Débit, latence, saturation pools, backpressure.	k6/Gatling/JMeter.
Résilience	Pannes, latence, duplications, restarts.	Toxiproxy/Chaos Mesh/kill pods.
Sécurité	AuthZ, IDOR, injection, secrets, dépendances.	SAST/SCA + tests dédiés.

15.2 Jeux de tests critiques

- 100 requêtes concurrentes avec la même Idempotency-Key -> une seule opération financière.
- Deux remboursements concurrents -> jamais de sur-remboursement.
- Webhook provider dupliqué 100 fois -> une seule transition/écriture métier.
- Crash juste après commit DB -> retry client retourne la ressource canonique.
- Kafka indisponible -> paiement commité, outbox non perdue, publication ultérieure.
- Consumer redémarré après effet mais avant offset commit -> pas de double ledger.
- Migration schema compatible pendant rolling deployment.
- Secrets et tokens de paiement absents des logs, exceptions et traces.

15.3 Quality gates

Gate	Condition minimale
Build	Compilation + tests unitaires réussis.
Coverage	Seuil indicatif sur domaine critique, mais pas utilisé comme métrique unique de qualité.
Static analysis	Aucun finding critique non accepté.
Dependencies	Aucune vulnérabilité critique connue non traitée/justifiée.
Integration	Migrations + Testcontainers passent.
Contracts	OpenAPI/events compatibles.
Performance	Pas de régression p95 supérieure au budget défini.
Security	Tests d'autorisation multi-tenant et idempotence verts.

16. CI/CD, infrastructure et Kubernetes

16.1 Pipeline de livraison



Figure 16 - Pipeline CI/CD cible.

16.2 Conteneurisation

- Images OCI minimales, non-root, avec health/readiness endpoints.
- Multi-stage build ou buildpacks ; image reproductible et scannée.
- Aucun secret dans l'image ou les layers.
- Tag immutable par commit SHA + tag de release.
- SBOM générée lorsque l'outillage le permet.

16.3 Kubernetes

- Deployments avec requests/limits CPU et mémoire.
- Readiness distincte de liveness ; startup probe si démarrage long.
- PodDisruptionBudget sur services critiques.
- Horizontal Pod Autoscaler basé sur métriques appropriées.
- NetworkPolicy lorsque l'environnement le permet.
- ConfigMaps pour configuration non sensible ; secrets via solution dédiée.
- Rolling update avec maxUnavailable faible pour le cœur paiement.
- Jobs/CronJobs pour migrations ou réconciliation seulement avec stratégie sûre.

16.4 Infrastructure as Code

- Terraform gère réseau, cluster, base, cache, Kafka, registry, IAM et observabilité selon le cloud retenu.
- État Terraform distant et verrouillé.
- Séparation dev/staging/prod par workspace ou structure de modules explicite.
- Principe least privilege pour IAM et service accounts.
- Destruction de l'environnement de démo doit être documentée pour maîtriser les coûts.

16. CI/CD, infrastructure et Kubernetes - architecture de déploiement

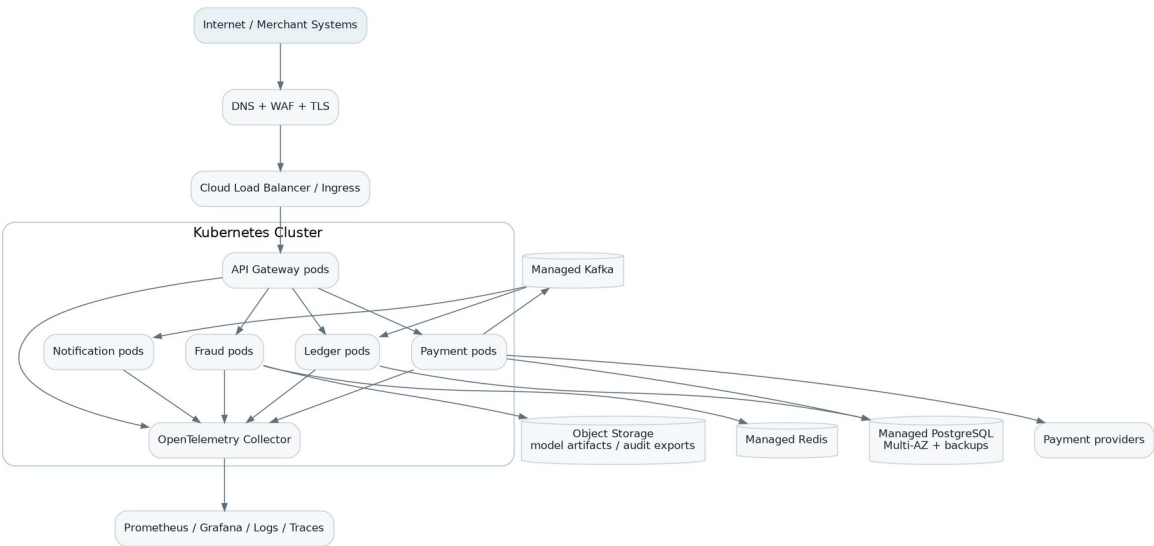


Figure 17 - Architecture de déploiement cloud cible.

16.5 Stratégie de déploiement

Environnement	But	Données
Local	Développement rapide avec Docker Compose/Testcontainers.	Synthétiques uniquement.
CI	Tests éphémères isolés.	Fixtures synthétiques.
Staging	E2E, sandbox provider, load tests contrôlés.	Synthétiques, jamais données réelles.
Production demo	Démonstration publique limitée, quotas et monitoring stricts.	Synthétiques / sandbox ; aucune carte réelle stockée.

16.6 Migrations zéro/low downtime

1. Déployer d'abord un changement de schéma backward compatible (ajout nullable, nouvelle table, index concurrent lorsque supporté).
2. Déployer l'application capable de lire/écrire ancien et nouveau format si nécessaire.
3. Backfiller hors chemin critique avec contrôle de charge.
4. Basculer les lectures vers le nouveau format.
5. Retirer l'ancien champ/chemin dans une release ultérieure après vérification.

17. Exigences non fonctionnelles

ID	Catégorie	Exigence	Priorité
NFR-001	Intégrité	Aucun double débit ou sur-remboursement issu d'un replay/concurrency couvert par le système.	P0
NFR-002	Disponibilité	Les lectures restent disponibles lors d'une panne de notification ; les paiements dégradent proprement selon provider/fraud.	P1
NFR-003	Performance	Budgets p95/p99 documentés et mesurés ; aucune requête DB N+1 critique.	P1
NFR-004	Scalabilité	Services stateless horizontalement scalables ; partitions/topics et pool DB dimensionnés.	P1
NFR-005	Sécurité	Authentification, autorisation multi-tenant, secrets, chiffrement, audit et anti-replay.	P0
NFR-006	Observabilité	Toute requête critique corrélable par traceId ; dashboards et alertes disponibles.	P1
NFR-007	Maintenabilité	Modules/services avec frontières, tests d'architecture et ADR.	P1
NFR-008	Déployabilité	Build et déploiement reproductibles depuis un commit versionné.	P1
NFR-009	Récupération	Backups restaurables testés ; procédure RTO/RPO documentée.	P1
NFR-010	Portabilité	Local dev via containers ; cloud ciblé sans coupler le domaine aux APIs cloud.	P2
NFR-011	Traçabilité	Événements financiers, décisions fraude et actions admin auditable.	P0
NFR-012	Coût	Architecture de démo dimensionnée pour coûts maîtrisés ; composants managed seulement quand justifiés.	P2

17.1 Budgets de capacité pour tests

Les chiffres suivants sont des cibles de test du projet et doivent être remplacés par des mesures obtenues. Ils servent à organiser la démarche de performance.

Phase	Charge cible de laboratoire	But
Monolithe local	100 req/s soutenues sur endpoints simples	Identifier limites DB/JVM et établir baseline.
Staging	300-500 req/s mix lecture/écriture synthétique	Mesurer p95/p99 et saturation.
Stress	Augmenter jusqu'à dégradation contrôlée	Trouver le bottleneck et vérifier backpressure.
Soak	Charge moyenne plusieurs dizaines de minutes/heures	Détecter fuites, accumulation, lag et dérive mémoire.

18. Roadmap de développement par niveaux

Règle de progression

Chaque niveau se termine par une démonstration, des tests, un ADR ou une note de conception, et une version taggée. Ne pas passer au niveau suivant tant que les invariants du niveau courant ne sont pas prouvés.

Niveau	Thème	Livrables essentiels
Niveau 0	Bootstrap & standards	Repo, Java, Spring Boot, Gradle/Maven, quality gates, Docker local, README, ADR template.
Niveau 1	Modular monolith	Merchant, Payment Intent, REST, validation, erreurs, Flyway, PostgreSQL, tests.
Niveau 2	Transactions & concurrence	Isolation, locks, idempotency durable, refund correctness, performance SQL.
Niveau 3	Payment provider	Adapter Stripe sandbox + MockProvider, webhooks entrants signés, state machine.
Niveau 4	Security	OIDC/JWT, RBAC, API keys, scopes, audit log, rate limiting.
Niveau 5	Ledger	Double-entry simplifié, invariant balance, réconciliation de base.
Niveau 6	Outbox & Kafka	Transactional outbox, topics, idempotent consumers, retry/DLQ, schema versioning.
Niveau 7	Extraction services	Extraire Notification/Fraud puis Ledger/Payment selon ADR, database ownership.
Niveau 8	Redis & résilience	Feature cache, rate limit, timeouts, circuit breakers, chaos scenarios.
Niveau 9	Fraud ML	ONNX/PMML, feature builder, versioning, audit, explainability reasons.
Niveau 10	Observabilité	OTel, Prometheus, Grafana, structured logs, traces, SLO/alerts/runbooks.
Niveau 11	Performance & chaos	k6/Gatling, Toxiproxy/Chaos Mesh, profiling JVM/SQL, failure reports.
Niveau 12	Cloud & Kubernetes	Terraform, managed DB/cache/Kafka, K8s, TLS/WAF, backups.
Niveau 13	Production hardening	Zero-downtime migrations, security review, DR test, cost controls, public demo.
Niveau 14	Portfolio release	Architecture docs, diagrams, benchmark report, incident postmortem, CV bullets, demo script.

18.1 Conditions de passage

- Tests critiques verts et reproductibles.
- Aucune dette P0 connue non documentée.
- Documentation d'API et architecture mise à jour.
- Un scénario d'échec significatif est démontré.
- Mesures ou preuves objectives disponibles (test, dashboard, trace, benchmark, migration, pipeline).

19. Critères d'acceptation et Definition of Done

19.1 Acceptation fonctionnelle globale

AC	Scénario	Résultat attendu
AC-01	Créer puis confirmer un paiement sandbox	Ressource SUCCEEDED/FAILED cohérente, provider_ref persistée, audit et événement générés.
AC-02	Rejouer 50 fois la même création avec même clé	Une seule Payment Intent canonique ; réponses cohérentes.
AC-03	Deux remboursements concurrents au-delà du disponible	Au maximum le montant remboursable est accepté.
AC-04	Envoyer plusieurs fois le même webhook provider	Une seule transition métier / aucun double ledger.
AC-05	Couper Kafka pendant un paiement	Paiement DB conservé ; événement en outbox puis livré à la reprise.
AC-06	Couper Notification Service	Paiement non annulé ; backlog visible et retry/DLQ.
AC-07	Fraud score BLOCK	Provider non appelé ; décision audité.
AC-08	Clé API d'un autre marchand	Accès refusé ; aucune fuite inter-tenant.
AC-09	Déployer une nouvelle version	Rolling deploy sans interruption perceptible sur lecture ; rollback possible.
AC-10	Rechercher un paiement	Trace complète : requête, provider, fraude, ledger, événements, webhooks.

19.2 Definition of Done d'une user story

- Critères d'acceptation automatisés ou reproductibles.
- Tests unitaires/intégration appropriés.
- Migrations backward compatibles si données affectées.
- AuthN/AuthZ et audit évalués.
- Metrics/logs/traces ajoutés pour le comportement important.
- Documentation OpenAPI mise à jour.
- Pas de secret, donnée carte ou PII inutile dans logs.
- Pipeline CI vert et artefact versionné.
- ADR ajouté si décision architecturale significative.

20. Risques, décisions et dette technique

20.1 Registre de risques initial

Risque	Impact	Prob.	Mitigation
Complexité trop élevée trop tôt	Élevé	Élevée	Commencer modular monolith, gates par niveau.
Double débit / état ambigu provider	Critique	Moyenne	Idempotence, provider refs, réconciliation, state machine.
Sur-remboursement concurrent	Critique	Moyenne	Locks/version + contraintes + tests de concurrence.
Perte d'événement Kafka	Élevé	Moyenne	Transactional outbox + retries + monitoring.
Double consommation	Élevé	Élevée	Consumers idempotents / processed_events.
Fuite de secrets	Critique	Faible-Moy.	Secrets manager, scans, logs redaction, rotation.
Coûts cloud excessifs	Moyen	Moyenne	Budgets, quotas, environnement éphémère, managed services limités.
Fraud model peu crédible	Moyen	Moyenne	Jeu de données public/synthétique, focus intégration/versioning plutôt que claims de précision.
K8s sans nécessité réelle	Moyen	Élevée	Introduire uniquement à la phase deployment/scaling et documenter le coût.
Documentation divergente	Moyen	Moyenne	Docs-as-code, PR checklist, ADR et diagrammes versionnés.

20.2 ADR minimum attendus

6. ADR-001 - Modular monolith avant microservices.
7. ADR-002 - PostgreSQL comme source de vérité.
8. ADR-003 - Stratégie d'idempotence et durée de rétention.
9. ADR-004 - State machine de paiement.
10. ADR-005 - Ledger append-only et modèle double entrée.
11. ADR-006 - Transactional outbox vs dual write.
12. ADR-007 - Kafka topic/partition strategy.
13. ADR-008 - Extraction des microservices.
14. ADR-009 - Usage de Redis et stratégie d'invalidation.
15. ADR-010 - Politique de dégradation du moteur fraude.
16. ADR-011 - Stratégie d'observabilité et SLO.
17. ADR-012 - Choix cloud/Kubernetes et maîtrise des coûts.

21. Livrables du repository et documentation

21.1 Arborescence cible

```

nexapay/
├── apps/                # monolith puis services extraits
├── libs/                # contrats / utilitaires strictement partagés
├── infra/
│   ├── docker/
│   ├── kubernetes/
│   └── terraform/
├── docs/
│   ├── architecture/
│   ├── adr/
│   ├── api/
│   ├── runbooks/
│   ├── threat-model/
│   └── benchmarks/
├── tests/
│   ├── e2e/
│   ├── load/
│   └── chaos/
├── observability/
│   ├── dashboards/
│   └── alerts/
├── .github/workflows/
├── docker-compose.yml
├── Makefile / Taskfile
├── README.md
├── CONTRIBUTING.md
└── SECURITY.md

```

21.2 Documents obligatoires

Document	Contenu
README	Vision, architecture courte, quickstart, démo, stack, liens.
architecture.md	Contexte, containers/components, data ownership, sync/async flows.
payment-flow.md	State machine, idempotence, provider ambiguity, refunds.
security.md	Threat model, auth, secrets, data classification, incident handling.
events.md	Catalogue Kafka, schemas, partition keys, retry/DLQ.
observability.md	SLI/SLO, dashboards, alerts, trace examples.
deployment.md	Local/staging/prod, IaC, migrations, rollback.
runbooks/*.md	Incidents fréquents et procédures.
benchmarks/*.md	Méthodologie, environnement, résultats et bottlenecks.
postmortems/*.md	Au moins un incident simulé documenté sans blâme.

21.3 Preuves à montrer en entretien

- Un diagramme d'architecture lisible et une explication de la progression monolithe -> services.
- Un test de concurrence de remboursement/idempotence.
- Une trace distribuée d'un paiement complet.
- Un dashboard de latence/taux d'erreur et Kafka lag.
- Un incident simulé et son postmortem.
- Un ADR expliquant pourquoi Kafka, Redis ou Kubernetes a été introduit.
- Un rapport de benchmark avant/après optimisation SQL ou cache.
- Un pipeline de déploiement reproductible.

22. Annexes techniques

22.1 Checklist avant mise en production de démonstration

- [] TLS actif et redirection HTTP->HTTPS.
- [] Aucun secret commité ; scanners secrets verts.
- [] Clés provider uniquement sandbox/test.
- [] Rate limits et quotas activés.
- [] Backups DB configurés et restauration testée.
- [] Dashboards et alertes disponibles.
- [] Health/readiness/liveness vérifiés.
- [] Migrations compatibles rolling deploy.
- [] Runbook rollback testé.
- [] DLQ et backlog outbox à zéro ou expliqués.
- [] Tests idempotence/concurrence verts.
- [] Audit multi-tenant validé.
- [] Logs vérifiés pour absence de données sensibles.
- [] Budget cloud et alertes coût configurés.
- [] README et diagrammes à jour.

22.2 Glossaire

Terme	Définition
Payment Intent	Agrégat représentant l'intention d'encaisser un montant avant résultat final.
Idempotence	Propriété garantissant que répéter une commande avec la même identité n'ajoute pas d'effet supplémentaire.
Outbox	Table transactionnelle contenant les événements à publier après commit local.
Inbox	Stockage permettant de déduplicer des messages entrants déjà traités.
Ledger	Journal immuable d'écritures financières.
SLO	Objectif de niveau de service mesurable à partir d'un SLI.
DLQ	Dead Letter Queue/Topic pour messages non traitables après retries.
RPO	Perte de données maximale acceptable exprimée en temps.
RTO	Temps maximal visé pour restaurer le service.
MFA	Authentification multi-facteurs.
OIDC	Couche d'identité basée sur OAuth 2.0.
ONNX	Format portable pour exécuter des modèles ML dans différents runtimes.

22. Annexes techniques - matrices de traçabilité

22.3 Matrice exigence -> preuve

Exigence	Preuve attendue
Idempotence paiement	Test concurrent + contrainte DB + capture trace/log.
Webhooks idempotents	Test 100 duplications + inbox/provider_event_id unique.
Outbox	Test Kafka down/recovery + lignes outbox + publication ultérieure.
Ledger équilibré	Property/integration tests : somme débits = crédits.
Multi-tenant isolation	Tests d'autorisation croisée entre deux marchands.
Fraud versioning	Décision persistée avec modelVersion/featureVersion/reasons.
Observabilité	Dashboard + exemple de trace d'un paiement.
Performance	Rapport k6/Gatling avec p95/p99 et environnement décrit.
Résilience	Scénarios Toxiproxy/Chaos et postmortem.
Déploiement	GitHub Actions + Terraform/K8s + rollback démontré.

22.4 Matrice module -> responsabilité

Module / service	Responsabilité principale	Données propriétaires
Merchant	Compte marchand, configuration, API keys metadata, webhook endpoints.	merchant, api_key, webhook_endpoint
Payment	Payment Intent, Payment, Refund, provider orchestration, idempotency.	payment_intent, payment, refund, idempotency_record
Fraud	Features, modèle, policy, décision risque.	fraud_decision, feature metadata
Ledger	Transactions et écritures financières immuables.	ledger_transaction, ledger_entry, account
Notification	Jobs de livraison, tentatives et DLQ.	delivery, delivery_attempt
Identity/Access	Rôles/scopes locaux et mapping IdP.	role_binding, permissions metadata
Outbox/Inbox	Fiabilité événements et déduplication.	outbox_event, processed_event/webhook_inbox

22. Annexes techniques - scénarios d'entretien

22.5 Questions auxquelles le système doit permettre de répondre

18. Pourquoi avoir commencé par un modular monolith plutôt que cinq microservices ?
19. Comment garantis-tu qu'un client qui retry après timeout n'est pas débité deux fois ?
20. Que se passe-t-il si PostgreSQL commit mais Kafka est indisponible ?
21. Comment empêches-tu deux remboursements concurrents de dépasser le montant capturé ?
22. Pourquoi le ledger est-il append-only ?
23. Quelle est ta clé de partition Kafka et quelle propriété cherches-tu à préserver ?
24. Comment un consumer Kafka gère-t-il les doublons ?
25. Que se passe-t-il si le service fraude est lent ou indisponible ?
26. Pourquoi Redis n'est-il jamais la source de vérité financière ?
27. Comment diagnostiques-tu un paiement lent dans cinq services ?
28. Quelle migration de base est sûre pendant un rolling deployment ?
29. Comment prouves-tu que ton API supporte une certaine charge ?
30. Quel est ton mécanisme de rotation d'API keys et secrets webhook ?
31. Comment réduis-tu le périmètre de données carte et les risques associés ?
32. Quel incident as-tu simulé, comment l'as-tu détecté et corrigé ?

22.6 Définition de la version portfolio finale

Release Portfolio

La release portfolio n'est pas la version contenant le plus de technologies. C'est la version pour laquelle chaque composant a une justification, chaque invariant critique possède une preuve, le déploiement est reproductible et les scénarios de panne sont documentés.

22.7 Références techniques de travail

- Spring Framework / Spring Boot / Spring Security - documentation officielle correspondant à la version retenue.
- PostgreSQL - documentation officielle sur transactions, isolation, locking, indexation et EXPLAIN.
- Apache Kafka - documentation officielle sur producer/consumer semantics, partitions, offsets et transactions.
- Stripe - documentation API et webhook signing en environnement sandbox.
- OAuth 2.0 et OpenID Connect - standards d'authentification/autorisation applicables.
- RFC 9457 - Problem Details for HTTP APIs pour la standardisation des erreurs.
- OpenAPI Specification - contrat et documentation de l'API publique.
- OWASP ASVS / OWASP API Security - exigences et tests de sécurité applicative.
- PCI DSS - bonnes pratiques et réduction de périmètre ; ne jamais présenter le projet comme certifié sans audit officiel.
- OpenTelemetry - instrumentation distribuée, contexte de trace et conventions sémantiques.
- Kubernetes et Terraform - documentation officielle pour déploiement et Infrastructure as Code.

Synthèse finale - Architecture et ordre d'apprentissage

Le projet doit progresser selon l'ordre suivant afin de maximiser l'apprentissage et de conserver une architecture défendable :

```

Java / Spring Boot / SQL
↓
Modular Monolith + REST + Tests
↓
Transactions + Concurrency + Idempotency
↓
Payment Provider + Webhooks + Refunds
↓
Security + API Keys + OIDC/RBAC
↓
Ledger + Reconciliation
↓
Transactional Outbox + Kafka
↓
Microservice Extraction
↓
Redis + Resilience
↓
Fraud ML (ONNX/PMML)
↓
OpenTelemetry + Metrics + Traces
↓
Load/Chaos Testing
↓
CI/CD + Terraform + Kubernetes + Cloud
↓
Production Hardening + Portfolio Evidence
  
```

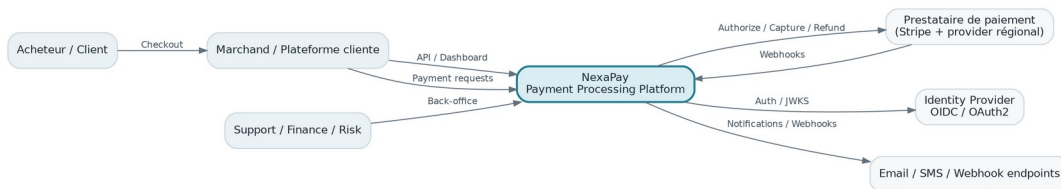


Figure 18 - NexaPay reste une plateforme centrée sur la fiabilité du paiement, et non sur la quantité de technologies.

Fin du cahier des charges - Version 1.0. Toute modification de périmètre significative doit être reflétée dans une version ultérieure et, si nécessaire, dans un ADR.